

□ Les Méta-Classes en Python

Le support est une propriété de **Bechir Bejaoui**

1 □ Comprendre la hiérarchie —tout est objet

Pour comprendre les méta-classes, il faut d'abord comprendre cette hiérarchie à 3 niveaux.

```
class Voiture:
    marque = "Renault"

v = Voiture()

# Niveau 1 : v est une instance de Voiture
print(type(v))          # <class '__main__.Voiture'>

# Niveau 2 : Voiture est une instance de... type !
print(type(Voiture))    # <class 'type'>

# Niveau 3 : type est une instance de Lui-même
print(type(type))       # <class 'type'>

<class '__main__.Voiture'>
<class 'type'>
<class 'type'>
```

Hiérarchie complète :

v	→ instance de	→ Voiture	(classe ordinaire)
Voiture	→ instance de	→ type	(méta-classe par défaut)
type	→ instance de	→ type	(se crée lui-même)

Objet → Classe → Méta-classe

- **v est une instance** de Voiture —c'est familier
- **Voiture est une instance** de type —c'est la révélation : une classe est un objet comme les autres
- **type** est la **méta-classe par défaut** —c'est elle qui fabrique toutes les classes Python
- **méta-classe** = classe dont les instances sont elles-mêmes des classes

Une **métaclass** sert à **contrôler ou modifier la création des classes elles-mêmes**.

classe normale → crée des **objets**

métaclasses → crée des **classes**

À quoi ça sert réellement

1. Imposer des règles aux classes

Vérifier automatiquement que certaines méthodes existent.

Exemple : forcer toutes les classes d'un framework à définir une méthode save.

2. Modifier automatiquement la classe

Ajouter des attributs ou méthodes au moment où la classe est créée.

Utilisé dans des frameworks.

Voici un exemple simple et concret.

L'idée :

- la métaclasses vérifie que toute classe qui l'utilise contient bien une méthode save
- si save manque, la création de la classe échoue immédiatement
- la métaclasses ajoute automatiquement une méthode to_dict pour sérialiser l'objet

Créer le fichier exemple_métaclasses.py :

```
class ModelMeta(type):  
  
    def __new__(mcls, name, bases, namespace):  
  
        if name != "BaseModel" and "save" not in namespace:  
            raise TypeError(f"La classe {name} doit définir une méthode save()")  
  
        def to_dict(self):  
            return self.__dict__  
  
        namespace["to_dict"] = to_dict  
  
        return super().__new__(mcls, name, bases, namespace)
```

Classe de base :

```
class BaseModel(metaclass=ModelMeta):  
    pass
```

Classe valide :

```
class User(BaseModel):

    def __init__(self, nom, age):
        self.nom = nom
        self.age = age

    def save(self):
        print("Sauvegarde utilisateur")
```

Utilisation :

```
u = User("Bechir", 40)
```

```
u.save()
print(u.to_dict())
```

Résultat :

```
Sauvegarde utilisateur
{'nom': 'Bechir', 'age': 40}
```

La métaclasse :

- vérifie que la classe possède save()
- ajoute automatiquement la méthode to_dict().

2 □ type comme méta-classe par défaut

La classe type a deux usages : vérifier le type d'un objet, et **créer dynamiquement une classe**.

```
# Usage 1 : vérifier le type (connu)
print(type(42))          # <class 'int'>
print(type("bonjour"))  # <class 'str'>
```

```
# Usage 2 : créer une classe dynamiquement
# type(nom, bases, attributs)
#   ↑       ↑       ↑
#  str      tuple  dict
```

```
Personne = type(
    "Personne",          # nom de la classe
    (),                  # classes parentes (tuple vide = pas d'héritage)
    {
        "espece": "humain",
        "saluer": lambda self: print(f"Bonjour, je suis {self.espece}")
    })
```

```

    }
)

p = Personne()
print(p.espece)    # humain
p.saluer()         # Bonjour, je suis humain
print(type(p))     # <class '__main__.Personne'>

humain
Bonjour, je suis humain
<class '__main__.Personne'>

```

- **type(nom, bases, dict)** : forme à 3 arguments —crée une vraie classe Python à la volée
- **"Personne"** : le nom qui apparaîtra dans `type(obj)` et `repr(obj)`
- **()** : tuple des classes parentes —(object,) implicitement, comme toute classe Python 3
- **dict** : le corps de la classe —attributs et méthodes

Ce qui se passe quand on écrit `class < nom de classe> :`

Ces deux formes sont EXACTEMENT équivalentes :

```

class Personne:                                # forme habituelle
    espece = "humain"

Personne = type("Personne", (), {              # ce que Python fait en interne
    "espece": "humain"
})

```

- **class <nom de classe> :** est du sucre syntaxique —Python appelle `type(...)` en coulisses
- **comprendre ça, c'est comprendre les méta-classes** : si on remplace `type` par notre propre classe, on contrôle la création des classes

3 □ Un deuxième exmple de méta-classe

Objectif de la métaclasse :

1. **forcer toutes les classes à définir un attribut `table`**
2. **enregistrer automatiquement les classes dans un registre**

C'est exactement ce que font beaucoup de frameworks (ORM, plugins, serializers...).

Créer la métaclasse :

```

class ModelMeta(type):
    registry = {}

    def __new__(mcls, name, bases, namespace):
        if name != "Model" and "table" not in namespace:
            raise TypeError(f"La classe {name} doit définir l'attribut 'table'")

        cls = super().__new__(mcls, name, bases, namespace)

        if name != "Model":
            ModelMeta.registry[name] = cls

        return cls

```

Classe de base :

```

class Model(metaclass=ModelMeta):
    pass

```

Deux classes valides :

```

class User(Model):
    table = "users"

class Product(Model):
    table = "products"

```

4 □ Anatomie d'une méta-classe —les 3 méthodes clés

Une méta-classe peut intercepter 3 moments distincts dans la vie d'une classe.

```

class TraceMeta(type):
    def __new__(cls, name, bases, namespace):
        # ① appelé quand Python CRÉE la classe (définition)
        print(f" 1. __new__ : construction de '{name}'")
        return super().__new__(cls, name, bases, namespace)

    def __init__(cls, name, bases, namespace):
        # ② appelé juste après __new__, pour INITIALISER la classe
        print(f" 2. __init__ : initialisation de '{name}'")
        super().__init__(name, bases, namespace)

```

```

def __call__(cls, *args, **kwargs):
    # ③ appelé quand on INSTANCIE la classe (Chien())
    print(f" 3. __call__ : instantiation de '{cls.__name__}'")
    return super().__call__(*args, **kwargs)

print("=== Définition de la classe ===")
class Chien(metaclass=TraceMeta):
    def __init__(self, nom):
        self.nom = nom

print("\n=== Instanciation ===")
rex = Chien("Rex")
print(f"  rex.nom = {rex.nom}")

=== Définition de la classe ===
1. __new__ : construction de 'Chien'
2. __init__ : initialisation de 'Chien'

=== Instanciation ===
3. __call__ : instantiation de 'Chien'
rex.nom = Rex

Ordre d'exécution :

class Chien(metaclass=TraceMeta): → Meta.__new__ puis Meta.__init__
rex = Chien("Rex") → Meta.__call__ → Chien.__new__ → Chien.__init__

• Meta.__new__ : construit l'objet classe —reçoit le namespace (dictionnaire du corps de la classe)
• Meta.__init__ : initialise la classe après sa construction —même signature que __new__
• Meta.__call__ : intercepte chaque Chien(...) —c'est ici qu'on peut bloquer ou modifier l'instanciation
• name, bases, namespace : les 3 paramètres que Python passe toujours —nom de la classe, tuple des parents, dict des attributs

```

5 □ Cas d'usage concret —validation automatique

La méta-classe la plus utile en pratique : **imposer des règles** à toutes les classes d'une hiérarchie, sans que les développeurs aient à y penser.

```

class ModelMeta(type):
    """

```

*Méta-classe pour un mini ORM.
 Impose que chaque modèle déclare __tablename__.*

```
def __new__(cls, name, bases, namespace):
    # On ignore la classe de base elle-même
    if name != "BaseModel":
        if "__tablename__" not in namespace:
            raise TypeError(
                f"La classe '{name}' doit définir __tablename__"
            )
        return super().__new__(cls, name, bases, namespace)

class BaseModel(metaclass=ModelMeta):
    """Classe de base — toutes les classes qui en héritent sont validées."""
    pass

# 📄 Classe correcte
class User(BaseModel):
    __tablename__ = "users"

class Commande(BaseModel):
    __tablename__ = "commandes"

print("User créé 📄")
print("Commande créée 📄")

# 📄 Classe incorrecte — erreur à la DÉFINITION, pas à l'exécution
try:
    class Produit(BaseModel):
        pass # oubli de __tablename__
except TypeError as e:
    print(f"Erreur : {e}")

User créé 📄
Commande créée 📄
Erreur : La classe 'Produit' doit définir __tablename__
```

- **l'erreur survient à la définition** — pas quand on utilise la classe — les problèmes sont détectés le plus tôt possible
- **if name != "BaseModel"** : exclut la classe de base elle-même — sinon elle échouerait aussi
- **namespace** : le dict du corps de la classe avant sa création — on peut le lire, le modifier, le compléter
- **use case réel** : Django ORM, SQLAlchemy, marshmallow utilisent exacte-

ment ce pattern

6 □ Inspection du namespace

La méta-classe reçoit le namespace —le dictionnaire de tout ce qui est défini dans le corps de la classe. On peut l'inspecter ou le modifier avant que la classe soit créée.

```
class InspectMeta(type):

    def __new__(cls, name, bases, namespace):
        # Compter les méthodes et attributs
        methodes = [k for k, v in namespace.items()
                     if callable(v) and not k.startswith("__")]
        attributs = [k for k, v in namespace.items()
                     if not callable(v) and not k.startswith("__")]

        print(f"Classe '{name}' :")
        print(f"  - Méthodes : {methodes}")
        print(f"  - Attributs : {attributs}")

        return super().__new__(cls, name, bases, namespace)
```

```
class Service(metaclass=InspectMeta):
    version = "1.0"
    actif = True

    def demarrer(self):
        pass

    def arreter(self):
        pass
```

```
Classe 'Service' :
- Méthodes : ['demarrer', 'arreter']
- Attributs : ['version', 'actif']
```

- **namespace** contient tout ce qui est dans le corps de la classe : méthodes, attributs, `__doc__`, `__module__`...
- **les clés `__dunder__`** : on les filtre avec `startswith("__")` pour ne garder que le contenu métier
- **on peut modifier namespace avant** `super().__new__()` —c'est là que toute la magie opère (ajout de méthodes, renommage...)

7 □ Injection automatique de méthodes

Une méta-classe peut **ajouter des méthodes** à toutes les classes d'une hiérarchie —sans que le développeur ait à les écrire.

```
class AutoMeta(type):
    """Injecte to_dict() dans toute classe qui ne l'a pas définie."""

    def __new__(cls, name, bases, namespace):

        # Injecter to_dict() si absente
        if "to_dict" not in namespace:
            def to_dict(self):
                return {k: v for k, v in self.__dict__.items()
                        if not k.startswith("_")}
            namespace["to_dict"] = to_dict

        return super().__new__(cls, name, bases, namespace)

class BaseModel(metaclass=AutoMeta):
    pass

class Client(BaseModel):
    def __init__(self, nom, email):
        self.nom = nom
        self.email = email

class Produit(BaseModel):
    def __init__(self, nom, prix):
        self.nom = nom
        self.prix = prix

c = Client("Bechir", "b@mail.com")
p = Produit("Clavier", 99.9)

print(c.to_dict())  # méthode injectée automatiquement
print(p.to_dict())  # idem

{'nom': 'Bechir', 'email': 'b@mail.com'}
{'nom': 'Clavier', 'prix': 99.9}
```

- `namespace["to_dict"] = to_dict` : on ajoute la méthode directement dans le dict avant création —la classe la recevra comme si elle l'avait écrite
- `if "to_dict" not in namespace` : respecte le choix du développeur —si la classe définit sa propre version, on ne l'écrase pas
- `self.__dict__` : attributs de l'instance —on filtre les `_privés` pour ne garder que les attributs publics

8 □ `__new__` vs `__init__` dans la méta-classe

Les deux méthodes sont appelées lors de la définition d'une classe. Leurs rôles sont distincts.

```
class DemoMeta(type):

    def __new__(cls, name, bases, namespace):
        # __new__ : CONSTRUIT l'objet classe
        # C'est ici qu'on peut modifier namespace AVANT création
        print(f" __new__ : '{name}' — namespace modifiable ici")
        namespace["cree_par"] = "DemoMeta" # ajout avant création
        return super().__new__(cls, name, bases, namespace)

    def __init__(cls, name, bases, namespace):
        # __init__ : INITIALISE la classe déjà construite
        # La classe existe, on peut y ajouter des attributs
        print(f" __init__ : '{name}' — classe accessible via cls")
        cls.meta_version = "1.0" # ajout après création
        super().__init__(name, bases, namespace)

class MaClasse(metaclass=DemoMeta):
    pass

print(f"\ncree_par : {MaClasse.cree_par}")
print(f"meta_version: {MaClasse.meta_version}")

__new__ : 'MaClasse' — namespace modifiable ici
__init__ : 'MaClasse' — classe accessible via cls

cree_par : DemoMeta
meta_version: 1.0
```

	<code>__new__</code>	<code>__init__</code>
Quand	Avant création	Après création

	<code>__new__</code>	<code>__init__</code>
Accès à la classe	Non (pas encore créée)	Oui via <code>cls</code>
Modifier namespace	☐ Oui	☐ Trop tard
Ajouter attribut	<code>namespace["x"] = ...</code>	<code>cls.x = ...</code>
Cas d'usage	Valider / transformer le corps	Enrichir la classe créée

9 ☐ Registre automatique de classes

Pattern très courant : **enregistrer automatiquement** toutes les sous-classes dans un registre centralisé —utile pour les plugins, les handlers, les routes.

```
class PluginMeta(type):
    """Enregistre automatiquement chaque plugin créé."""
    _registre = {}

    def __new__(cls, name, bases, namespace):
        nouvelle_classe = super().__new__(cls, name, bases, namespace)

        # On n'enregistre pas la classe de base elle-même
        if bases:
            cls._registre[name] = nouvelle_classe
            print(f"☐ Plugin enregistré : '{name}'")

        return nouvelle_classe

class BasePlugin(metaclass=PluginMeta):
    def executer(self):
        raise NotImplementedError

class PluginEmail(BasePlugin):
    def executer(self):
        print("☐ Envoi d'email")

class PluginSMS(BasePlugin):
    def executer(self):
        print("☐ Envoi de SMS")

class PluginSlack(BasePlugin):
```

```

def executer(self):
    print(" 📧 Message Slack")

print(f"\nPlugins disponibles : {list(PluginMeta._registre.keys())}")

# Utilisation du registre : exécuter tous les plugins
print("\nExécution de tous les plugins :")
for nom, classe in PluginMeta._registre.items():
    classe().executer()

📧 Plugin enregistré : 'PluginEmail'
📧 Plugin enregistré : 'PluginSMS'
📧 Plugin enregistré : 'PluginSlack'

Plugins disponibles : ['PluginEmail', 'PluginSMS', 'PluginSlack']

Exécution de tous les plugins :
📧 Envoi d'email
📧 Envoi de SMS
📧 Message Slack

```

- **if bases:** : bases est vide pour BasePlugin (pas de parent explicite)
—on exclut ainsi la classe de base du registre
- **cls._registre** : dict de classe sur la méta-classe —partagé par toutes les méta-classes du même type
- **intérêt** : le développeur n'a **rien à faire** —créer la classe suffit à l'enregistrer automatiquement

❑ Méta-classe vs décorateur de classe

Les décorateurs et les métaclases interviennent à **des moments différents**.

Le **décorateur** agit après la création de la classe.

Python fait ceci :

- **Étape 1** : créer la classe
- **Étape 2** : appeler le décorateur

La **méta-classe** agit pendant la création de la classe.

Version décorateur

Le décorateur vérifie **après que la classe existe**.

```
def verifier_run(cls):

    if not hasattr(cls, "run"):
        raise TypeError("La classe doit définir une méthode run()")

    return cls
```

```
@verifier_run
class Tache:

    def run(self):
        print("execution")
```

Si quelqu'un oublie run :

```
@verifier_run
class MauvaiseTache:
    pass
```

Erreur :

TypeError: La classe doit définir une méthode run()

Le décorateur agit **après que Python ait déjà créé la classe.**

Version métaclasse

La métaclasse agit **pendant la création de la classe**

```
class VerifierMeta(type):

    def __new__(mcls, name, bases, namespace):

        if name != "TacheBase" and "run" not in namespace:
            raise TypeError("La classe doit définir une méthode run()")

        return super().__new__(mcls, name, bases, namespace)
```

Classe de base :

```
class TacheBase(metaclass=VerifierMeta):
    pass
```

Classe correcte :

```
class Tache(TacheBase):

    def run(self):
        print("execution")
```

Classe incorrecte :

```
class MauvaiseTache(TacheBase):  
    pass
```

Différence visible

Python crée la classe

↓

le décorateur vérifie/modifie la classe

Python demande à la métaclasse de créer la classe

↓

la métaclasse vérifie/modifie AVANT création

Guide de choix :

DÉCORATEUR vs MÉTA-CLASSE	
Décorateur de classe	Méta-classe
Modifie après création Simple, lisible Manuel (@decorator) type(cls) inchangé Cas : 1-2 classes	Agit pendant la création Plus puissant, plus complexe Automatique (toutes les sous-cls) type(cls) = la méta-classe Cas : toute une hiérarchie

Règle pratique :

Si ça marche avec un décorateur → utiliser le décorateur

Si on a besoin d'héritage automatique → utiliser une méta-classe

1 □1 □ Ordre d'exécution complet

Lors de la définition ET de l'instanciation d'une classe avec méta-classe, l'ordre exact est :

```
class Meta(type):  
  
    def __new__(cls, name, bases, namespace):  
        print(f" 1. Meta.__new__ : construction de '{name}'")  
        return super().__new__(cls, name, bases, namespace)  
  
    def __init__(cls, name, bases, namespace):  
        print(f" 2. Meta.__init__ : initialisation de '{name}'")
```

```

    super().__init__(name, bases, namespace)

def __call__(cls, *args, **kwargs):
    print(f" 3. Meta.__call__ : quelqu'un fait {cls.__name__}()")
    return super().__call__(*args, **kwargs)

class Chien(metaclass=Meta):

    def __new__(cls):
        print(f" 4. Chien.__new__ : création de l'instance")
        return super().__new__(cls)

    def __init__(self):
        print(f" 5. Chien.__init__ : initialisation de l'instance")

print("— Définition de la classe —")
# (steps 1 et 2 se produisent ici, à la lecture du class)

print("\n— Instanciation —")
rex = Chien()

```

— Définition de la classe —

1. Meta.__new__ : construction de 'Chien'
2. Meta.__init__ : initialisation de 'Chien'

— Instanciation —

3. Meta.__call__ : quelqu'un fait Chien()
4. Chien.__new__ : création de l'instance
5. Chien.__init__ : initialisation de l'instance

- **étapes 1-2** se produisent à la **définition** de la classe —dès que Python lit le bloc class Chien:
- **étape 3** se produit à chaque **instanciation** —Meta.__call__ est le gardien de la création d'instances
- **étapes 4-5** sont les méthodes habituelles de la classe —déclenchées par super().__call__() dans la méta-classe

1 □2 □ Cas réel —Singleton via méta-classe

Le Singleton garantit **une seule instance** d'une classe. La méta-classe est l'endroit idéal pour l'implémenter —propre, réutilisable.

```

class SingletonMeta(type):
    """Garantit qu'une seule instance est créée par classe."""
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            print(f" Première création de {cls.__name__}")
            cls._instances[cls] = super().__call__(*args, **kwargs)
        else:
            print(f" Instance existante retournée pour {cls.__name__}")
        return cls._instances[cls]

class ConfigApp(metaclass=SingletonMeta):
    def __init__(self):
        self.debug = False
        self.version = "1.0"

class ConnexionDB(metaclass=SingletonMeta):
    def __init__(self):
        self.url = "postgresql://localhost/db"

# Test
c1 = ConfigApp()
c2 = ConfigApp()
c3 = ConfigApp()

print(f"\nc1 is c2 : {c1 is c2}") # True
print(f"c1 is c3 : {c1 is c3}") # True

c1.debug = True
print(f"c2.debug : {c2.debug}") # True — même objet

db1 = ConnexionDB()
db2 = ConnexionDB()
print(f"\ndb1 is db2 : {db1 is db2}") # True

Première création de ConfigApp
Instance existante retournée pour ConfigApp
Instance existante retournée pour ConfigApp

c1 is c2 : True
c1 is c3 : True
c2.debug : True

```


Première création de ConnexionDB
Instance existante retournée pour ConnexionDB

db1 is db2 : True

- **__instances__** : dict de classe sur la méta-classe —clé = la classe, valeur = l'unique instance
 - **Meta.__call__** est l'endroit parfait : il s'exécute avant `__new__` et `__init__` —on peut intercepter avant même que l'objet soit créé
 - **réutilisable** : SingletonMeta s'applique à ConfigApp, ConnexionDB et n'importe quelle autre classe
-

1 3 3 Conflit de méta-classes comment le résoudre

Si une classe hérite de deux parents ayant des méta-classes différentes, Python refuse.

```
class M1(type): pass
class M2(type): pass
```

```
class A(metaclass=M1): pass
class B(metaclass=M2): pass
```

3 Conflit : A utilise M1, B utilise M2 — Python ne sait pas laquelle choisir

```
try:
    class C(A, B): pass
except TypeError as e:
    print(f"Conflit : {e}")
```

3 Solution : créer une méta-classe combinée

```
class MCombinee(M1, M2): pass # hérite des deux
```

```
class C(A, B, metaclass=MCombinee): pass # plus de conflit
print("C créé sans conflit 3")
print(f"type(C) = {type(C)}")
```

Conflit : metaclass conflict: the metaclass of a derived class must be a (non-strict) subclass of the metaclasses of all its bases
TypeError...

```
C créé sans conflit 3
type(C) = <class '__main__.MCombinee'>
```

- **le message d'erreur** dit exactement ce qu'il faut faire : la méta-classe de C doit être une sous-classe des méta-classes de ses parents
 - **MCombinee(M1, M2)** : hérite des deux → satisfait la contrainte pour A (M1) et pour B (M2)
 - **cas fréquent en pratique** : combiner une méta-classe custom avec ABCMeta
-

1 □4 □ ABCMeta —méta-classe des classes abstraites

ABC utilise déjà une méta-classe en interne : ABCMeta. Quand on écrit `class X(ABC)`, on utilise une méta-classe sans le savoir.

```
from abc import ABC, ABCMeta, abstractmethod

# ABC = sucre syntaxique pour metaclass=ABCMeta
class Animal(ABC):                                # équivaut à : class Animal(metaclass=ABCMeta)

    @abstractmethod
    def parler(self):
        """Chaque animal doit implémenter parler()."""
        pass

    def respirer(self):                            # méthode concrète : partagée par tous
        print(" □ Respiration")

# □ Classe concrète — implémente parler()
class Chien(Animal):
    def parler(self):
        print(" □ Wouf !")

# □ Classe incomplète — pas d'instanciation possible
try:
    a = Animal()
except TypeError as e:
    print(f"Erreur : {e}")

# □ Instanciation normale
rex = Chien()
rex.parler()
rex.respirer()

# La preuve : ABCMeta est bien une méta-classe
```

```
print(f"\ntype(Animal) = {type(Animal)}") # <class 'abc.ABCMeta'>
print(f"type(Chien) = {type(Chien)}") # <class 'abc.ABCMeta'>
```

Erreur : Can't instantiate abstract class Animal with abstract method parler
 ❑ Wouf !
 ❑ Respiration

```
type(Animal) = <class 'abc.ABCMeta'>
type(Chien) = <class 'abc.ABCMeta'>
```

- **ABCMeta** : méta-classe fournie par Python qui surveille l'implémentation de toutes les méthodes `@abstractmethod`
- **ABC** : classe de commodité équivalente à `metaclass=ABCMeta` —la forme recommandée au quotidien
- **type(Animal) = ABCMeta** : preuve que ABC utilise bien une méta-classe sous le capot

1 ❑5 ❑ Bonnes pratiques —quand utiliser les méta-classes ?

❑ Cas légitimes pour une méta-classe :

- ```
#
1. Valider des conventions sur toute une hiérarchie
2. Enregistrer automatiquement des sous-classes
3. Injecter des méthodes communes
4. Implémenter Singleton
5. Construire un ORM ou framework
```

# ❑ À éviter :

- ```
# - Si un décorateur de classe suffit → utiliser le décorateur
# - Si __init_subclass__ suffit → l'utiliser (plus simple, Python 3.6+)
```

— Alternative moderne : `__init_subclass__` —————

```
class BaseModel:
```

```
    def __init_subclass__(cls, **kwargs):
        """Appelé automatiquement à chaque sous-classe créée."""
        super().__init_subclass__(**kwargs)
        if not hasattr(cls, "__tablename__"):
            raise TypeError(f"'{cls.__name__}' doit définir __tablename__")
        print(f"❑ Modèle '{cls.__name__}' validé")
```

Même comportement que `ModelMeta` — sans méta-classe !

```
class User(BaseModel):
```

```

__tablename__ = "users"

try:
    class Produit(BaseModel):
        pass
except TypeError as e:
    print(f"Erreur : {e}")

```

❑ Modèle 'User' validé
 Erreur : 'Produit' doit définir __tablename__

QUAND UTILISER QUOI ?	
@decorateur	Modification simple d'une classe
__init_subclass__	Valider/enregistrer des sous-cls
Méta-classe	Contrôle total (namespace, type...)

Règle : utiliser la solution la plus simple qui résout le problème.
 Les méta-classes en dernier recours.

- **__init_subclass__** : introduit en Python 3.6 —couvre 80% des cas d'usage des méta-classes avec beaucoup moins de complexité
- **méta-classe** : nécessaire quand on veut modifier le namespace **avant** la création, ou changer le type de la classe
- **"Simple is better than complex"** (Zen of Python) —si **__init_subclass__** ou un décorateur suffisent, ne pas aller vers les méta-classes

❑ Récapitulatif —les 5 points clés

type fabrique toutes les classes · **méta-classe** = classe dont les instances sont des classes · **__new__** intercepte la définition, **__call__** intercepte l'instanciation · **namespace** = dict du corps de la classe, modifiable avant création · **préférer __init_subclass__** pour les cas simples avant d'utiliser une méta-classe

20 QUESTIONS INTELLIGENTES —MÉTA-CLASSES

Format : Énoncé → Solution Chaque solution contient : explication en tête, scripts enchaînés, démonstration observable.

1) Pourquoi une classe est-elle instance de type ?

Énoncé

Montrer que les classes sont créées par une méta-classe.

Solution

```
"""
Explication
- Toute classe est une instance de 'type' (ou d'une méta-classe personnalisée).
- La création suit : Meta.__new__ -> Meta.__init__.
"""

class A:
    pass

print("type(A) :", type(A))          # <class 'type'>
print("isinstance(A, type) :", isinstance(A, type))

# Preuve dynamique : créer une classe via type
B = type("B", (), {"x": 10})
b = B()
print("B.x :", b.x)
print("type(B) :", type(B))
```

2) Quand new d'une méta-classe est-il exécuté ?

Énoncé

Tracer la création d'une classe.

Solution

```
"""
Explication
- __new__ de la méta-classe s'exécute au moment de la définition de la classe.
"""

class TraceMeta(type):
    def __new__(mcls, name, bases, namespace):
        print("Meta.__new__ ->", name)
        return super().__new__(mcls, name, bases, namespace)

class C(metaclass=TraceMeta):
```

```

        pass

print("Classe C prête")

```

3) Différence entre new et init dans une méta-classe ?

Énoncé

Montrer l'ordre réel d'exécution.

Solution

```

"""
Explication
- __new__ construit la classe.
- __init__ initialise la classe.
- Ordre : __new__ -> __init__.
"""

class M(type):
    def __new__(mcls, name, bases, namespace):
        print("Meta new")
        return super().__new__(mcls, name, bases, namespace)

    def __init__(cls, name, bases, namespace):
        print("Meta init")
        super().__init__(name, bases, namespace)

class D(metaclass=M):
    pass

```

4) À quoi sert call dans une méta-classe ?

Énoncé

Intercepter l'instanciation.

Solution

```

"""
Explication
- __call__ de la méta-classe contrôle l'instanciation.
- Il encapsule : instance = __new__ + __init__.

```

```

"""

class CallMeta(type):
    def __call__(cls, *args, **kwargs):
        print("Meta __call__")
        instance = super().__call__(*args, **kwargs)
        print("Instance créée :", instance)
        return instance

class E(metaclass=CallMeta):
    def __init__(self):
        print("Init E")

e = E()

```

5) Pourquoi prepare existe ?

Énoncé

Contrôler le namespace avant création.

Solution

```

"""
Explication
- __prepare__ permet de définir le type de mapping utilisé pour le namespace.
- Utile pour préserver l'ordre ou injecter un dict personnalisé.
"""

class PrepMeta(type):
    @classmethod
    def __prepare__(mcls, name, bases):
        print("Prepare namespace")
        return {}

class F(metaclass=PrepMeta):
    x = 1

```

6) Pourquoi injecter dynamiquement une méthode ?

Énoncé

Ajouter to_dict automatiquement.

Solution

```
"""
Explication
- La méta-classe modifie le namespace avant création.
"""

class AutoDict(type):
    def __new__(mcls, name, bases, namespace):
        if "to_dict" not in namespace:
            def to_dict(self):
                return self.__dict__
            namespace["to_dict"] = to_dict
        return super().__new__(mcls, name, bases, namespace)

class User(metaclass=AutoDict):
    def __init__(self, name):
        self.name = name

u = User("Ali")
print(u.to_dict())
```

7) Comment imposer une convention métier ?

Énoncé

Forcer la présence d'un attribut obligatoire.

Solution

```
"""
Explication
- Lever une erreur si un attribut attendu manque.
"""

class ModelMeta(type):
    def __new__(mcls, name, bases, namespace):
        if name != "BaseModel" and "__tablename__" not in namespace:
            raise TypeError("Attribut __tablename__ obligatoire")
        return super().__new__(mcls, name, bases, namespace)

class BaseModel(metaclass=ModelMeta):
    pass

class Client(BaseModel):
```



```
__tablename__ = "clients"

print("Client validé")
```

8) Pourquoi une méta-classe peut servir de registre ?

Énoncé

Enregistrer automatiquement toutes les classes.

Solution

```
"""
Explication
- Enregistrer chaque classe créée dans une liste.
"""

class RegistryMeta(type):
    registry = []

    def __new__(mcls, name, bases, namespace):
        cls = super().__new__(mcls, name, bases, namespace)
        if name != "Base":
            mcls.registry.append(cls)
        return cls

class Base(metaclass=RegistryMeta):
    pass

class A(Base): pass
class B(Base): pass

print("Registry :", RegistryMeta.registry)
```

9) Pourquoi un conflit de méta-classe survient ?

Énoncé

Héritage multiple avec méta-classes incompatibles.

Solution

```
"""
Explication
- Si deux parents ont des méta-classes différentes,
  Python doit résoudre une méta-classe commune.
"""

class M1(type): pass
class M2(type): pass

class A(metaclass=M1): pass
class B(metaclass=M2): pass

# class C(A, B): pass # déclenche TypeError
```

10) Comment résoudre un conflit de méta-classe ?

Énoncé

Créer une méta-classe combinée.

Solution

```
"""
Explication
- La méta-classe enfant doit hériter des deux méta-classes.
"""

class M1(type): pass
class M2(type): pass

class Combined(M1, M2): pass

class A(metaclass=M1): pass
class B(metaclass=M2): pass

class C(A, B, metaclass=Combined): pass

print("C créé sans conflit")
```

11) Pourquoi ABCMeta est une méta-classe ?

Énoncé

Comprendre abstractmethod.

Solution

```
"""
Explication
- ABCMeta contrôle l'instanciation si méthodes abstraites non implémentées.
"""

from abc import ABCMeta, abstractmethod

class Base(metaclass=ABCMeta):
    @abstractmethod
    def run(self):
        pass

class Impl(Base):
    def run(self):
        print("OK")

Impl().run()
```

12) Pourquoi dataclass n'est pas une méta-classe ?

Énoncé

Comparer décorateur et méta-classe.

Solution

```
"""
Explication
- dataclass est un décorateur de classe.
- Il modifie la classe après création.
"""

from dataclasses import dataclass

@dataclass
class P:
    x: int
```

```
        y: int

print(type(P)) # reste <class 'type'>
```

13) Comment empêcher l'ajout d'attributs ?

Énoncé

Injection automatique de **slots**.

Solution

```
"""
Explication
- Injecter __slots__ dans namespace.
"""

class SlotMeta(type):
    def __new__(mcls, name, bases, namespace):
        namespace["__slots__"] = ("x", "y")
        return super().__new__(mcls, name, bases, namespace)

class Point(metaclass=SlotMeta):
    pass

p = Point()
p.x = 1
```

14) Peut-on transformer toutes les méthodes ?

Énoncé

Auto-wrap des méthodes.

Solution

```
"""
Explication
- Parcourir namespace et décorer chaque fonction.
"""

def log(fn):
    def wrapper(*args, **kwargs):
```

```

        print("LOG:", fn.__name__)
        return fn(*args, **kwargs)
    return wrapper

class AutoLog(type):
    def __new__(mcls, name, bases, namespace):
        for k, v in namespace.items():
            if callable(v):
                namespace[k] = log(v)
        return super().__new__(mcls, name, bases, namespace)

class Service(metaclass=AutoLog):
    def run(self):
        print("Running")

Service().run()

```

15 □ Ordre réel d'exécution lors d'une instantiation contrôlée par méta-classe

Énoncé

Préciser et démontrer l'ordre exact des appels lorsque l'on instancie une classe contrôlée par une méta-classe.

Solution

```

"""
Explication
Lorsqu'on exécute instance = Classe():

1) Meta.__call__()
2) Classe.__new__()
3) Classe.__init__()

La méta-classe contrôle entièrement l'instanciation.
"""

class Meta(type):
    def __call__(cls, *args, **kwargs):
        print("1 - Meta.__call__")
        return super().__call__(*args, **kwargs)

class A(metaclass=Meta):

```

```

def __new__(cls):
    print("2 - Classe.__new__")
    return super().__new__(cls)

def __init__(self):
    print("3 - Classe.__init__")

```

a = A()

Sortie observée :

- 1 - Meta.call
 - 2 - Classe.new
 - 3 - Classe.init
-

16 □ Comment implémenter un Singleton propre via méta-classe ?

Énoncé

Créer un Singleton robuste garantissant une seule instance.

Solution

```

"""
Explication
La méta-classe stocke les instances déjà créées.
Si la classe a déjà été instanciée, elle retourne la même instance.
"""

```

```

class SingletonMeta(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            print("Création nouvelle instance")
            cls._instances[cls] = super().__call__(*args, **kwargs)
        else:
            print("Réutilisation instance existante")
        return cls._instances[cls]

class Database(metaclass=SingletonMeta):
    def __init__(self):

```

```

        print("Initialisation Database")

db1 = Database()
db2 = Database()

print("Même instance :", db1 is db2)

```

17 □ Comment empêcher l'instanciation d'une classe ?

Énoncé

Interdire toute création d'instance via la méta-classe.

Solution

```

"""
Explication
On bloque __call__ pour empêcher toute instanciation.
"""

class NoInstanceMeta(type):
    def __call__(cls, *args, **kwargs):
        raise TypeError("Instanciation interdite")

class Utility(metaclass=NoInstanceMeta):
    pass

try:
    u = Utility()
except TypeError as e:
    print("Erreur :", e)

```

18 □ Comment transformer automatiquement toutes les méthodes d'une classe ?

Énoncé

Appliquer un wrapper automatique sur toutes les méthodes.

Solution

```
"""
Explication
On parcourt le namespace avant création.
On remplace chaque fonction par une version décorée.
"""

def tracer(fn):
    def wrapper(*args, **kwargs):
        print(f"[TRACE] {fn.__name__}")
        return fn(*args, **kwargs)
    return wrapper

class AutoTraceMeta(type):
    def __new__(mcls, name, bases, namespace):
        for key, value in namespace.items():
            if callable(value):
                namespace[key] = tracer(value)
        return super().__new__(mcls, name, bases, namespace)

class Service(metaclass=AutoTraceMeta):
    def run(self):
        print("Execution run")

    def stop(self):
        print("Execution stop")

s = Service()
s.run()
s.stop()
```

19 □ Comment injecter automatiquement un champ horodaté dans toutes les instances ?

Énoncé

Ajouter automatiquement un attribut `created_at` à chaque instance créée.

Solution

```
"""
Explication
On surcharge __call__ pour injecter un champ après instantiation.
"""
```

```
import datetime
```

```
class TimestampMeta(type):
    def __call__(cls, *args, **kwargs):
        instance = super().__call__(*args, **kwargs)
        instance.created_at = datetime.datetime.now()
        return instance
```

```
class Order(metaclass=TimestampMeta):
    def __init__(self, id):
        self.id = id
```

```
o = Order(10)
print("ID :", o.id)
print("Created at :", o.created_at)
```

20 □ Comment implémenter un mini ORM simplifié via méta-classe ?

Énoncé

Créer un mécanisme qui :

- Enregistre les champs déclarés
- Génère automatiquement une méthode save()
- Maintient un registre global des modèles

Solution

```
"""
Explication
La méta-classe :
- détecte les attributs déclarés comme champs
- les stocke dans _fields
"""
```

- ajoute automatiquement une méthode save()
 - enregistre les modèles dans un registre
 """

```
class Field:
    def __init__(self, type_):
        self.type_ = type_

class ORMMeta(type):
    registry = {}

    def __new__(mcls, name, bases, namespace):

        fields = {}

        for key, value in namespace.items():
            if isinstance(value, Field):
                fields[key] = value.type_

        namespace["_fields"] = fields

        def save(self):
            data = {k: getattr(self, k) for k in self._fields}
            print(f"Saving into table {name}:", data)

        namespace["save"] = save

        cls = super().__new__(mcls, name, bases, namespace)

        if name != "BaseModel":
            mcls.registry[name] = cls

        return cls

class BaseModel(metaclass=ORMMeta):
    pass

class User(BaseModel):
    id = Field(int)
    name = Field(str)

    def __init__(self, id, name):
        self.id = id
```

```
self.name = name
```

```
u = User(1, "Ali")  
u.save()
```

```
print("Registry :", ORMMeta.registry)  
print("Champs détectés :", User._fields)
```